OOPs Project Documentation

Topic: Cloud Native Disaster Mapping

Presented to: Dr. Nikita Tiwari

Presented By:

1. **Anmol Gaba** 590013947
2. **Aditya Hari Prakash Sharma** 590012826
3. **Taraksh Jigar Raval** 590012998
4. **Prakash Kumar** 590016782

OBJECTIVES:

The main objective of this project is to develop a **Cloud-Native Disaster Management System** that enables real-time reporting, monitoring, and coordination of disaster-related incidents, resources, and volunteers.

- To provide a platform for reporting disaster incidents.
- To manage alerts and emergency notifications.
- To track affected areas using location data.

- To manage resources such as food, medical aid, and rescue equipment.
- To coordinate volunteers for disaster response.
- To ensure scalable and efficient backend services using cloud-native architecture.
- To demonstrate real-world implementation of Object-Oriented Programming concepts.

OOPS PRINCIPLES AND JAVA CONCEPTS USED:

OOPS PROJECT USED:

Encapsulation:
Data for entities like Alert, Incident, Resource, User, and Volunteer are stored in separate model classes with controlled access using getters and setters.

Inheritance:
Spring Boot annotations like @RestController and @Service provide inherited framework behaviour.

Polymorphism:
Spring Boot annotations like @RestController and @Service provide inherited framework functionalities.

Abstraction:
Frontend interacts with backend through REST APIs while internal logic is hidden.

JAVA CONCEPTS USED:

- Spring Boot Framework (REST API Development)
- Hibernate (JPA)
- MVC Architecture
- Dependency Injection
- Collections Framework

DATA FLOW OF APPLICATION:

1. User registers and logs into the system.
2. User reports a disaster incident with location and details.
3. Incident data is stored in the database.
4. Admin or system generates alerts for affected areas.
5. Resources (food, medical aid, etc.) are added and tracked.
6. Volunteers register and update availability.
7. System retrieves and displays all data for monitoring and response.

Flow:

User → Incident Reporting → Alert Generation → Resource Allocation → Volunteer Coordination → Database Storage

IMPLEMENTATION:

The project is fully implemented using Spring Boot with REST APIs.

The system follows a three-tier architecture:

1. **Presentation Layer**
   - HTML, CSS, JavaScript
   - User interface
2. **Application Layer**
   - Spring Boot REST APIs
   - Business logic
3. **Data Layer**
   - MySQL Database
   - Hibernate (JPA)

SYSTEM INTERFACE (FRONTEND MODULES):

The system provides a web-based interface for managing disaster operations. It is developed using HTML, CSS, and JavaScript and communicates with the backend using REST APIs.

Dashboard Module

- Displays incidents, alerts, resources, and volunteers
- Shows recent incidents
- Map visualization using Leaflet
- Real-time updates using WebSocket

Incident Module

- Users can report incidents
- Includes location, severity, and description
- Search and filter functionality

Alerts Module

- Create and manage emergency alerts
- Displays severity and status

Resource Module

- Add and manage resources
- Track availability

Volunteer Module

- Register volunteers

- Track skills and availability

Authentication Module

- User login and registration
- Session handled using local storage

APPLICATION LAYER:

The Application Layer is responsible for handling the core business logic of the system. It is developed using Spring Boot and exposes RESTful APIs for communication between the frontend and the database.

This layer processes user requests, performs validations, and manages data flow across different modules such as incidents, alerts, resources, and volunteers.

Core Components:

- **Controllers:**
  Handle incoming API requests (e.g., IncidentController, AlertController, ResourceController).
- **Services:**
  Contain business logic and act as an intermediary between controllers and repositories.
- **Repositories:**
  Interface with the database using Spring Data JPA.

Functionality Provided:

- User authentication (login/register)
- Incident reporting and retrieval
- Alert creation and management
- Resource allocation and tracking
- Volunteer registration and availability tracking

DATABASE DESIGN:

The system uses a relational database (MySQL) to store and manage disaster-related data. Each module is represented as a separate table, ensuring modularity and efficient data handling.

1. User Table

Stores registered user details.

- **id** (Primary Key)
- firstName
- lastName
- email
- password
- profileImage

✓ Used for authentication and user management.

2. Incident Table

Stores all reported disaster incidents.

- **id** (Primary Key)
- title
- type
- severity
- description
- location
- latitude
- longitude
- status
- date

✓ Core table of the system for disaster tracking.

3. Alert Table

Stores emergency alerts issued by the system.

- **id** (Primary Key)
- title
- severity
- status

- message
- targetArea
- createdAt
- ✓ Used for notifying users about dangerous situations.

4. Resource Table

Stores information about available resources.

- **id** (Primary Key)
- name
- type
- quantity
- status
- contact
- location
- ✓ Helps manage disaster response resources.

5. Volunteer Table

Stores registered volunteers.

- **id** (Primary Key)
- name

- skill
- availability
- ✓ Used for assigning volunteers to disaster tasks.

COMPLETED FEATURES:

- User Registration and Login System
- Incident Reporting with location data
- Alert Management System
- Resource Management (availability tracking)
- Volunteer Registration and Availability Tracking
- Database Integration using MySQL
- RESTful API-based communication

TECHNOLOGIES USED:

Frontend (UI):

- HTML
- CSS
- JavaScript
- Leaflet Map API

Backend:

- Java
- Spring Boot
- REST Controllers

Database:

- MySQL

Tools Used:

- Eclipse / Spring Tool Suite
- MySQL Workbench

FUTURE SCOPE:

- Real-time location tracking using maps
- Live disaster heatmap visualization
- Integration with government emergency systems
- AI-based disaster prediction
- Cloud deployment using AWS or Kubernetes
- Mobile application for quick reporting

CONCLUSION:

The Cloud Native Disaster Mapping System demonstrates the implementation of a scalable disaster management platform using Java, Spring Boot, and MySQL.

The system efficiently manages incidents, alerts, resources, and volunteers through REST APIs, enabling better coordination during emergencies.

This project reflects real-world backend architecture and strengthens understanding of cloud-native design and Object-Oriented Programming concepts.